

JOURNAL 3

Frontend Implementation

Smart City Civic Complaint and Management System

Project	Smart City Civic Complaint and Management System
Development Stage	Frontend Prototype Implementation
Technology Used	React, Vite, React Router, JavaScript, CSS
State Management	React Context API
Student	Achintya Maurya

1. Objective

The objective of this development phase was to implement the frontend prototype of the Smart City Civic Complaint and Management System using React. The frontend was developed to provide an interactive interface for citizens and department staff and to demonstrate the main complaint management workflow before backend and database integration.

The work focused on application setup, navigation, role-based interfaces, complaint submission and tracking, shared complaint state, officer processing, and frontend integration.

2. Development Environment and Technologies

The frontend was developed using the following technologies:

- **React** – used for developing reusable user interface components.
- **Vite** – used for creating and running the React development project.
- **JavaScript** – used for frontend application logic.
- **React Router** – used for navigation between different application pages.
- **CSS** – used for styling dashboards, forms, cards, navigation bars, buttons, and status indicators.
- **React Context API** – used for maintaining shared complaint information between frontend components.

3. Frontend Project Setup

The React frontend project was created using Vite. The project structure was organized into separate folders for pages and application context so that the frontend could be developed in a modular manner.

The main frontend structure includes separate page groups for:

- Citizen
- Department Officer
- Field Worker
- Administrator

The development server was configured and the application was successfully tested in the browser during implementation.

4. Application Routing

React Router was integrated into the application to provide navigation between the different frontend interfaces.

The following major routes were implemented:

- Login
- Register
- Citizen Dashboard
- Submit Complaint
- My Complaints
- Complaint Details
- Officer Dashboard
- Complaint Review
- Worker Dashboard
- Worker Task Details
- Administrator Dashboard

The routing structure allows each role to access its corresponding frontend interface without creating separate applications.

5. Login and Registration Interface

A login interface was developed as the entry point of the application. The interface contains fields for email and password along with a role-selection field.

The available prototype roles are:

- Citizen
- Department Officer
- Field Worker

The selected role determines which dashboard is opened after login.

A registration page was also implemented and connected to the login page.

At this stage, authentication is implemented only as a frontend prototype. Actual authentication and authorization will be handled during backend integration.

6. Citizen Dashboard

The Citizen Dashboard was developed as the main interface for citizens. It provides access to the major complaint-related operations.

The dashboard contains:

- Complaint statistics.
- Recent complaint information.
- Navigation to submit a complaint.
- Navigation to view submitted complaints.
- Complaint tracking access.
- Logout navigation.

The dashboard was designed to provide a simple interface through which a citizen can submit and monitor civic complaints.

7. Complaint Submission

A dedicated *Submit Complaint* page was implemented for reporting civic issues.

The complaint form contains the following fields:

- Complaint Category
- Complaint Description
- Location
- Photograph

The available categories in the frontend prototype include:

- Garbage / Waste
- Street Light
- Road Damage
- Water Supply
- Drainage
- Other

The citizen enters the description of the civic issue and provides the location where the issue has occurred. A photograph can also be selected through the image upload field.

After submission, a temporary complaint ID is generated and a success message is displayed to the citizen.

8. My Complaints

The *My Complaints* page was implemented to allow citizens to view their submitted complaints.

Each complaint card displays:

- Complaint ID
- Complaint title
- Complaint category

- Location
- Submission date
- Current complaint status

A *View Details* option is provided for opening the detailed information of an individual complaint.

9. Complaint Details

The *Complaint Details* page was implemented to display detailed information about a complaint selected by the citizen.

The page contains:

- Complaint title
- Complaint ID
- Category
- Location
- Submission date
- Current status
- Complaint description

A progress section was also included to represent the different stages of complaint processing.

10. Shared Complaint State

To allow different frontend roles to work with common complaint information, a shared state mechanism was implemented using the React Context API.

A file named `ComplaintContext.jsx` was created for maintaining the complaint collection.

The complaint object stores information such as:

- Complaint ID

- Title
- Category
- Location
- Date
- Citizen information
- Email
- Description
- Status
- Assigned worker
- Work progress
- Work notes
- Evidence
- Citizen verification status

Two main functions were implemented:

- `addComplaint()` – adds a new complaint to the shared complaint collection.
- `updateComplaint()` – updates information of an existing complaint.

The `ComplaintProvider` was connected to the main React application so that the complaint state could be accessed by different components.

11. Officer Dashboard Integration

The Officer Dashboard was connected with the shared complaint context.

Instead of using a separate static complaint list, the dashboard reads complaint information from the common complaint state.

Dynamic statistics were implemented for:

- Total Complaints
- Pending Verification

- In Progress
- Resolved

The dashboard also displays complaint information and provides a review option for each complaint.

12. Complaint Review and Verification

The Complaint Review page was integrated with the shared complaint state. The complaint ID from the selected route is used to identify the complaint that needs to be reviewed.

The officer can view:

- Complaint title
- Complaint ID
- Category
- Location
- Submission date
- Citizen name
- Citizen email
- Complaint description
- Current status

The following officer actions were implemented:

- **Verify Complaint**
- **Reject Complaint**
- **Assign Field Worker**

When the officer verifies a complaint, its status is updated in the shared frontend state. If the complaint is rejected, its status is changed to *Rejected*.

After verification, the officer can select a field worker and assign the complaint.

13. Worker Assignment Integration

The frontend was prepared so that a verified complaint can be assigned to a field worker. The officer can select a worker from the available worker list. Once the assignment is confirmed:

- The assigned worker is stored with the complaint.
- The complaint status changes to *Assigned*.
- The worker can access the corresponding task through the worker interface.

This integration connects the officer-side processing with the field-worker workflow through the shared frontend complaint state.

14. Frontend Integration and Testing

The different frontend components were tested together to verify that the application works as a single system.

The following activities were tested:

- Starting the React application through the Vite development server.
- Navigation between different application routes.
- Role-based dashboard navigation.
- Citizen complaint form interaction.
- Complaint listing and detail navigation.
- Officer complaint review.
- Complaint verification and rejection.
- Field-worker assignment.
- Updating complaint information through React Context.

During integration, a duplicate router configuration caused a routing error. This was resolved by maintaining a single `BrowserRouter` in `main.jsx` and keeping the application routes inside `App.jsx`.

15. Current Frontend Result

The frontend prototype has reached a functional multi-role stage. The Citizen, Department Officer, Field Worker, and Administrator interfaces have been implemented as separate frontend views.

The complaint information can be shared between the implemented interfaces through React Context, allowing the officer-side actions to update the common complaint state.

The frontend currently demonstrates the application workflow without requiring a backend server or database.

16. Current Limitations

The current frontend implementation has the following limitations:

- Complaint information is currently maintained in React Context and is reset when the application is refreshed.
- Login is currently a frontend prototype and does not perform real authentication.
- Photograph and evidence inputs are currently handled at the frontend level.
- Persistent storage has not yet been implemented.
- Backend API integration is not part of the current frontend stage.

17. Learning Outcomes

The frontend implementation provided practical understanding of:

- React component-based development.
- React Router and multi-page navigation.
- Role-based frontend organization.
- Interactive form handling in React.
- Shared state management using React Context API.
- Dynamic rendering based on complaint status.
- Frontend integration and debugging.
- Building an interactive prototype before backend integration.

18. Conclusion

The frontend implementation successfully established the user-facing structure of the Smart City Civic Complaint and Management System. The major citizen and officer interfaces, complaint handling functions, routing, and shared complaint state were implemented and tested.

The completed frontend provides the foundation for further integration with the remaining application modules and, subsequently, the backend API and database.